Composite Design Pattern

Category: Structural | **GoF Reference:** Gang of Four — Design Patterns (1994), Chapter 4

Table of Contents

Intent

The **Composite Design Pattern** lets you compose objects into tree structures to represent part-whole hierarchies, and then lets clients treat individual objects and compositions of objects **uniformly through a common interface**.

In plain terms: you define a single interface (the *Component*). Both a simple, indivisible object (the *Leaf*) and a complex container of objects (the *Composite*) implement that same interface. Client code never needs to know whether it is talking to a leaf or a composite — it always calls the same method on the same interface type, and the right behaviour happens automatically through polymorphism and recursion.

This eliminates branching logic in the caller. The caller does not ask "is this a file or a directory?" — it just calls `display()` and the object figures out what to do.

The Problem It Solves

Consider a file system. You have files and directories. Directories can contain files, and they can also contain other directories. The user — or the code — wants to display everything under a given root.

Without the Composite pattern, client code must distinguish between files and directories manually:

```
// Without Composite — ugly, brittle, hard to extend void displayItem(Object item) { if (item instanceof File) { File f = (File) item; System.out.println("File: " + f.getName()); } else if (item instanceof Directory) { Directory d = (Directory) item; System.out.println("Directory: " + d.getName()); for (Object child : d.getChildren()) { displayItem(child); // manual recursion scattered in client code } } // What if we add a SymbolicLink? Every caller must be updated. }
```

Problems with this approach:

- Every place in the codebase that handles file-system nodes needs its own `instanceof` chain.
- Adding a new node type (symbolic link, mount point, archive) requires hunting down every such chain and patching it.
- The recursion logic — walking into subdirectories — lives in the caller, not in the data structure. This makes callers complex and duplicates recursion logic across the codebase.
- Unit testing is harder because you cannot test a `Directory` node in isolation from the traversal logic.

With the Composite pattern, client code reduces to one line:

```
// With Composite — clean, uniform, extensible fileSystemComponent.display();
```

The `display()` call works whether `fileSystemComponent` is a single `File` or a deeply nested `Directory` tree. New node types (symbolic links, archives) just implement `FileSystemComponent` — no caller changes required.

When to Use

Use the Composite pattern when:

You have a tree (part-whole) hierarchy. The domain naturally decomposes into nodes that are either leaves (no children) or branches (containing other nodes of the same type).

You want clients to ignore the difference between individual objects and compositions. The client should call the same operation regardless of whether it is dealing with a leaf or a branch.

Adding new node types should not require changes to existing client code. Open/Closed Principle: open for extension (add a new leaf or composite class), closed for modification (callers do not change).

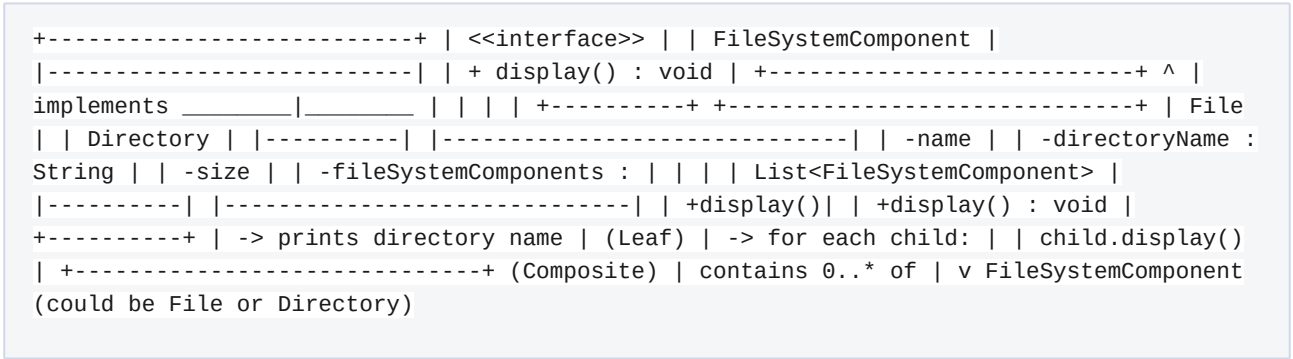
Recursive traversal should be encapsulated in the data structure, not in the caller. Each composite node is responsible for traversing its own children.

The depth of nesting is not known at compile time. The tree can be arbitrarily deep and can change at runtime (adding files, creating subdirectories).

Do not use Composite when:

- Every node in the hierarchy is a leaf (no containment). A plain list or array is simpler.
- You need to enforce strict constraints on which types of children a composite can hold (e.g., a `Form` can only contain `TextInput`, not arbitrary `Widget`). The pattern's uniformity works against such type-level restrictions.

Structure — ASCII UML



The critical observation in the UML is that `Directory` holds a `List<FileSystemComponent>` — a list of the *interface*, not the concrete `File` class. This means a `Directory` can hold any mix of `File` objects and other `Directory` objects (because `Directory` itself implements `FileSystemComponent`). This is what enables the recursive, arbitrarily deep tree structure.

Package Structure

```
com.design.patterns.composite ■■■ component ■ ■■■ FileSystemComponent.java (Component — the shared contract) ■■■ leaf ■ ■■■ File.java (Leaf — indivisible node) ■■■ composite ■ ■■■ Directory.java (Composite — branch node) ■■■ CompositeDesignPattern.java (Driver — client / entry point)
```

The three sub-packages exist for clarity and separation of concerns:

> **`component`** — Holds only the interface. This is the contract that every participant in the pattern must fulfill. It has no dependencies on `leaf` or `composite`. Other packages depend on it; it depends on nothing in the pattern.
>
> **`leaf`** — Holds concrete leaf implementations. Depends only on `component`. A leaf is by definition the terminating node — it has no children and does not import `composite`.
>
> **`composite`** — Holds the branch implementation. Depends on `component` (to hold `List<FileSystemComponent>`) but does not import `leaf` directly. It works with the interface, which means it can hold any future leaf type without modification.
>
> **Root package** — Holds only the driver / demo class. This is the only place where concrete types (`File`, `Directory`) are instantiated. It is the "composition root" — the one place in the application that knows about all the concrete implementations.

This layering enforces the Dependency Inversion Principle: high-level policy (`Directory`'s traversal logic) depends on the abstraction (`FileSystemComponent`), not on the concrete `File` class.

Line-by-Line Explanation of Every File

`FileSystemComponent.java` — Component Interface

```
package com.design.patterns.composite.component;
```

Declares the Java package. `component` is a sub-package of the pattern's root. Every class in this file belongs to this package. In a Maven project, this corresponds to the directory `src/main/java/com/design/patterns/composite/component/`.

```
public interface FileSystemComponent {
```

Declares a `public` Java interface named `FileSystemComponent`. It is `public` so that classes in other packages (`leaf`, `composite`, and the root driver) can implement or reference it. Using an `interface` rather than an abstract class is a deliberate choice: it allows leaf or composite classes to extend another class if needed (Java supports single inheritance but multiple interface implementation). It also expresses the intention clearly — this is a pure contract with no shared state.

```
void display();
```

The single method in the contract. All participants — `File` (leaf) and `Directory` (composite) — must provide a concrete `display()` implementation. The method returns `void` because its purpose is a side effect (printing to the console). In a production system this could return a `String` or write to a buffer, but `void` keeps the demo focused. The fact that this is the *only* method in the interface is intentional: the Composite pattern works best when the shared interface is minimal. A broader interface (e.g., adding `getSize()`, `getPermissions()`) would force leaf nodes to implement methods that may not make sense for them.

```
}
```

Closes the interface declaration.

`File.java` — Leaf

```
package com.design.patterns.composite.leaf;
```

Places `File` in the `leaf` sub-package. Leaf classes are the indivisible, terminal nodes of the tree. They have no children and do not hold a reference to any `FileSystemComponent`.

```
import com.design.patterns.composite.component.FileSystemComponent;
```

Imports the `FileSystemComponent` interface from the `component` package. This is the *only* import in this file. `File` does not know about `Directory`, and it does not need to. The leaf's dependency graph is minimal: it knows only about the contract it fulfills.

```
public class File implements FileSystemComponent {
```

Declares `File` as a concrete public class that fulfills the `FileSystemComponent` contract. The `implements FileSystemComponent` clause is the key statement — it enrolls this class into the Composite pattern. From this point, any variable of type `FileSystemComponent` can hold a `File` instance.

Note that `File` is a class name that shadows `java.io.File`. In a production codebase you would use a more domain-specific name (e.g., `TextFile`, `ImageFile`, or keep it in a namespace that avoids confusion), but for a pattern demonstration the collision is acceptable and understood from context.

```
private String name;
```

The file's name (e.g., `"Image1.png"`). It is `private` — correct encapsulation. No other class should directly access the raw field. If the name were needed externally, a `getName()` getter would be added, but since the only public operation is `display()`, a getter is not required here.

```
private int size;
```

The file's size in bytes (stored as a plain `int`). `private` for the same encapsulation reasons. Using `int` is fine for a demo; a production file system API would use `long` to handle files larger than ~2 GB (the `int` max value of 2,147,483,647 bytes).

```
public File(String name, int size) {
```

A public two-argument constructor. It is `public` so the driver (in the root package) can instantiate it. The constructor requires both `name` and `size` — there is no default constructor and no setters, which makes `File` effectively immutable after construction. Immutability is desirable for value-like objects in a tree.

```
this.name = name;
```

Assigns the constructor parameter `name` to the instance field `this.name`. The `this.` qualifier is necessary here because the constructor parameter and the field share the same identifier. Without `this.name =`, the assignment would be `name = name` — a no-op that writes back to the local parameter.

```
this.size = size;
```

Same pattern — assigns the constructor parameter `size` to the instance field `this.size`.

```
}
```

Closes the constructor body.

```
@Override
```

The `@Override` annotation tells the compiler that `display()` is intended to override a method declared in a supertype — in this case, the `FileSystemComponent` interface. If the signature does not match (e.g., due to a typo: `dispaly()`), the compiler emits an error instead of silently creating a new method that does not satisfy the contract. Always use `@Override` when implementing interface methods — it is a safety net.

```
public void display() {
```

Implements the `display()` method required by the `FileSystemComponent` contract. It is `public` because interface methods are implicitly public, and the implementing method must be at least as visible as the interface method.

```
System.out.println("File :" + name + " with size : " + size);
```

Prints the file's name and size to standard output. `name` and `size` refer to the instance fields. The string concatenation (`+`) creates a human-readable line such as `File :Image1.png with size : 1024`. Note the two spaces after `"File"` — this is intentional in the original source to create visual alignment with `"Directory : "` in the composite's output.

```
} }
```

Closes `display()` and the `File` class.

`Directory.java` — Composite

```
package com.design.patterns.composite.composite;
```

Places `Directory` in the `composite` sub-package. The package name is `composite.composite` which is slightly redundant; in practice you might name it `com.design.patterns.composite.branch` or `com.design.patterns.composite.node`, but the naming here makes the role explicit for a learning context.

```
import java.util.ArrayList; import java.util.List;
```

Imports the standard Java collection types. `List` is the interface used for the field declaration (programming to interfaces), and `ArrayList` is the concrete implementation used for initialization. Importing both is necessary because Java does not auto-import collection types.

```
import com.design.patterns.composite.component.FileSystemComponent;
```

Imports the shared component interface. `Directory` holds a `List<FileSystemComponent>` and iterates over it — the only external type it depends on. Crucially, `Directory` does not import `File`. It works with the interface, so it can hold any present or future `FileSystemComponent` implementation.

```
public class Directory implements FileSystemComponent {
```

Declares `Directory` as a concrete public class that also implements `FileSystemComponent`. This is the key insight of the Composite pattern: the composite (container) implements the same interface as the leaf. This is what allows a `Directory` to be held inside another `Directory`'s child list — because `Directory` is itself a `FileSystemComponent`.

```
private String directoryName;
```

The directory's display name (e.g., `"MyDirectory"`). `private` for encapsulation. Unlike the leaf fields, there is no `size` field here — directories in this model are purely structural containers, not measured by size.

```
List<FileSystemComponent> fileSystemComponents = new ArrayList<>();
```

This is the *defining field* of the Composite pattern. It is a list of the interface type `FileSystemComponent`, not of the concrete `File` type. This is what makes the pattern recursive and extensible:

- A `Directory` can hold `File` instances (because `File implements FileSystemComponent`).
- A `Directory` can hold other `Directory` instances (because `Directory implements FileSystemComponent`).
- A `Directory` can hold any future type that implements `FileSystemComponent` without any change to `Directory`.

The field is initialized to an empty `ArrayList<>()` as a defensive default. However, it is immediately overwritten in the constructor, so this initializer has no practical effect in this code — it just prevents the field from ever being `null` if the constructor logic were somehow bypassed (which cannot happen in this case, but it is a safe habit).

One issue worth noting: this field is package-private (no access modifier). In `com.design.patterns.composite.composite`, any other class in the same package can access it directly, bypassing encapsulation. It should be `private`. See Key Design Decisions for more.

```
public Directory(String directoryName, List<FileSystemComponent> fileSystemComponents) {
```

Public constructor taking the directory's name and its initial list of children. The `List<FileSystemComponent>` parameter type means the caller can pass any list of file-system nodes — mixing files and directories freely. This constructor-based approach (passing children at construction time) makes the object appear initialized from the start, which suits a demonstration, but it also means you cannot incrementally build the directory (no `addChild()` method). A production implementation would add a mutable `add(FileSystemComponent)` method.

```
this.directoryName = directoryName;
```

Assigns the constructor parameter to the instance field.

```
this.fileSystemComponents = fileSystemComponents;
```

Replaces the default empty `ArrayList` with the passed-in list. Note that this assignment stores the *reference* to the caller's list, not a copy. If the caller modifies the list after construction, the `Directory`'s children change too. A defensive copy (`new ArrayList<>(fileSystemComponents)`) would prevent this, but for a pattern demo the shared reference is fine.

```
}
```

Closes the constructor.

```
@Override public void display() {
```

Implements the `display()` method from `FileSystemComponent`. This implementation does two things: print the directory's own name, then delegate to each child's `display()`. The delegation is what makes the Composite pattern recursive.

```
System.out.println("Directory : " + this.directoryName);
```

Prints the directory header line (e.g., `Directory : MyDirectory`). The `this.` prefix is optional here — `directoryName` unambiguously refers to the instance field since there is no local variable of the same name — but it adds clarity in a teaching context.

```
fileSystemComponents.stream().forEach(FileSystemComponent::display);
```

This single line is the heart of the Composite pattern in action:

- `.stream()` — converts the `List<FileSystemComponent>` into a `Stream<FileSystemComponent>`, enabling use of the Stream API.
- `.forEach(...)` — iterates over every element in the stream and calls the given action on each.
- `FileSystemComponent::display` — a method reference. This is equivalent to the lambda `component -> component.display()`. It reads as "call `display()` on each `FileSystemComponent` in the stream."

Because each element is typed as `FileSystemComponent`, and both `File` and `Directory` implement that interface, Java dispatches `display()` polymorphically at runtime:

- If the element is a `File`, `File.display()` executes — it prints the file details and returns.
- If the element is a `Directory`, `Directory.display()` executes — it prints the directory name and then recurses into *its* children's `display()` calls.

This recursive delegation means a single top-level `display()` call traverses the entire tree, no matter how deep it goes, with no recursion logic in the client.

```
} }
```

Closes `display()` and the `Directory` class.

`CompositeDesignPattern.java` — Driver / Client

```
package com.design.patterns.composite;
```

The driver lives in the root package, not in any of the sub-packages. This is the composition root — the one place that knows about all concrete types.

```
import java.util.List;
```

Imports `java.util.List` to use `List.of(...)` for creating the immutable list of children.

```
import com.design.patterns.composite.component.FileSystemComponent;
```

Imports the component interface. The client declares its variables as `FileSystemComponent`, not as `File`. This is programming to the interface — the client code does not care about the concrete type after construction.

```
import com.design.patterns.composite.composite.Directory;
```

Imports the concrete `Directory` class. Needed only at the `new Directory(...)` construction site.

```
import com.design.patterns.composite.leaf.File;
```

Imports the concrete `File` class. Needed only at the `new File(...)` construction sites.

```
public class CompositeDesignPattern {
```

The main driver class. Named after the pattern for easy identification.

```
public static void main(String[] args) {
```

The JVM entry point. `static` because it is called without an instance. `String[] args` receives command-line arguments (unused here).

```
System.out.println("Composite Design Pattern");
```

Prints the pattern name to standard output as a banner. This is purely cosmetic — it identifies which pattern is being demonstrated when you run multiple pattern demos.

```
FileSystemComponent file1 = new File("Image1.png", 1024);
```

Creates the first leaf. The variable type is `FileSystemComponent` (the interface), not `File` (the concrete class). This is intentional: from this point forward, `file1` is treated as a `FileSystemComponent`. The concrete type is known only at the `new File(...)` call on the right-hand side. Using the interface type for the variable means you could swap in a different `FileSystemComponent` implementation without changing any downstream code.

The arguments `"Image1.png"` (name) and `1024` (size in bytes) describe a 1 KB PNG image file.

```
FileSystemComponent file2 = new File("Image2.png", 1024);
```

Creates the second leaf, identical in structure to `file1` but with a different name. Both are 1024 bytes.

```
Directory directory = new Directory("MyDirectory", List.of(file1, file2));
```

Creates the composite. Several things happen here:

- `new Directory("MyDirectory", ...)` — constructs a `Directory` named `"MyDirectory"`.
- `List.of(file1, file2)` — creates an immutable `List<FileSystemComponent>` containing both leaf nodes. Note that `file1` and `file2` are of type `FileSystemComponent`, so `List.of(file1, file2)` produces a `List<FileSystemComponent>` — exactly what `Directory`'s constructor expects.
- The variable type is `Directory` rather than `FileSystemComponent` because the driver wants to call `directory.display()` directly — though in this case `FileSystemComponent` would work equally well since `display()` is on the interface.

```
directory.display();
```

The single client call that triggers the entire tree traversal. Because `directory` is a `Directory`, `Directory.display()` runs. It prints the directory name, then iterates over the child list (which contains `file1` and `file2`), calling `display()` on each. The result is that the full tree is printed with no additional logic in the client.

```
} }
```

Closes `main()` and the driver class.

Execution Flow

Here is a step-by-step trace of what happens from the moment `main()` is called to the last line of output.

```
main() called ■ ■■■ System.out.println("Composite Design Pattern") ■ output: "Composite
Design Pattern" ■ ■■■ new File("Image1.png", 1024) → file1 = File{name="Image1.png",
size=1024} ■ ■■■ new File("Image2.png", 1024) → file2 = File{name="Image2.png",
size=1024} ■ ■■■ List.of(file1, file2) → immutable List<FileSystemComponent>[file1,
file2] ■ ■■■ new Directory("MyDirectory", list) → directory = Directory{ ■
directoryName="MyDirectory", ■ fileSystemComponents=[file1, file2] ■ } ■ ■■■
directory.display() ■ ■■■ System.out.println("Directory : MyDirectory") ■ output:
"Directory : MyDirectory" ■ ■■■
fileSystemComponents.stream().forEach(FileSystemComponent::display) ■ ■■■ [element 0 =
file1] → file1.display() (dispatched via FileSystemComponent::display) ■ ■■■
System.out.println("File :Image1.png with size : 1024") ■ output: "File :Image1.png with
size : 1024" ■ ■■■ [element 1 = file2] → file2.display() ■■■ System.out.println("File
:Image2.png with size : 1024") output: "File :Image2.png with size : 1024"
```

Final console output:

```
Composite Design Pattern Directory : MyDirectory File :Image1.png with size : 1024 File
:Image2.png with size : 1024
```

The key moment in this trace is the polymorphic dispatch inside `forEach`. The stream sees each element as `FileSystemComponent`. When it calls `display()` on an element, the JVM looks up the actual runtime type of that object and calls its implementation. For `file1` and `file2`, the runtime type is `File`, so `File.display()` runs. If one of those elements were instead a nested `Directory`, `Directory.display()` would run — which would in turn call `forEach` on its own children, and so on down the tree.

Real-World Use Cases

1. Operating System File System

The canonical example. Every OS file system is a Composite: files are leaves (they hold data, they have no children), directories are composites (they contain files and other directories). Operations like `du -sh` (disk usage), `find`, `ls -R`, and `cp -r` all traverse the tree uniformly — they do not special-case files vs. directories at the call site. The traversal logic is in the recursive descent, not in the command implementation.

2. GUI Widget Hierarchy (Swing, AWT, Android Views)

In a graphical user interface, a `Panel` (or `ViewGroup` in Android) contains other widgets: `Button`, `Label`, `TextField`, and other `Panel` objects. All of them implement a common `Component` interface with methods like `paint()`, `resize()`, `handleEvent()`. When the windowing system asks the root panel to repaint, it recursively repaints all children. Adding a new widget type (e.g., `ProgressBar`) requires only that it implement `Component` — no repaint logic changes.

3. Organization Chart (Manager/Employee Hierarchy)

A company org chart is a tree. At the leaves are individual contributors (employees with no direct reports). At the branches are managers, each managing a list of `Employee` nodes (which can themselves be managers). An operation like `getTotalHeadcount()` or `getTotalSalaryBudget()` naturally recurses: a leaf employee returns their own value, a manager returns their own value plus the sum over all direct reports. The Composite pattern models this without any type-checking in the payroll or reporting code.

4. HTML DOM Tree

Every HTML document is a tree of nodes. Text nodes (`#text`) and void elements (`<img>`, `<br>`) are leaves. Container elements (`<div>`, `<section>`, `<ul>`) are composites holding child nodes. Browser

rendering engines traverse this tree uniformly to compute layout, apply styles, paint pixels, and dispatch events. JavaScript's `element.querySelectorAll(selector)` is a Composite-pattern tree traversal — it works identically whether the root is a single `<span>` or the entire `<body>`.

5. Menu System with Submenus

A menu bar contains menu items. Some menu items open a submenu (composite), others trigger an action directly (leaf). The `render()` or `show()` operation on a menu item is uniform: a leaf item displays its label and registers a click handler; a composite item displays its label and, on hover, calls `render()` on each of its children. This allows arbitrarily deep submenu nesting with no special handling at the menu bar level.

6. Bill of Materials (BOM) in Manufacturing

A manufactured product is described by a Bill of Materials — a hierarchical list of components. A finished good (e.g., a laptop) is a composite containing sub-assemblies (motherboard, display panel, chassis) which are themselves composites containing individual parts (capacitors, screws, hinges) which are leaves. Operations like `getTotalCost()`, `getTotalWeight()`, and `getPartCount()` are all Composite-pattern recursive computations: each composite sums the result across its children, each leaf returns its own value.

Extending This Example — Nested Directories

The current demo has one level of nesting: a directory containing two files. The real power of the Composite pattern becomes apparent when you add a `Directory` *inside* another `Directory`. No changes to `FileSystemComponent`, `File`, or `Directory` are needed — you only change the driver.

```
public class CompositeDesignPattern { public static void main(String[] args) {
System.out.println("Composite Design Pattern – Nested"); // Leaves FileSystemComponent
img1 = new File("Image1.png", 1024); FileSystemComponent img2 = new File("Image2.png",
2048); FileSystemComponent doc1 = new File("Notes.txt", 512); FileSystemComponent doc2 =
new File("Report.pdf", 4096); // Inner composites Directory imagesDir = new
Directory("Images", List.of(img1, img2)); Directory docsDir = new Directory("Documents",
List.of(doc1, doc2)); // Outer composite containing inner composites Directory rootDir =
new Directory("Root", List.of(imagesDir, docsDir)); rootDir.display(); } }
```

Expected output:

```
Composite Design Pattern – Nested Directory : Root Directory : Images File :Image1.png
with size : 1024 File :Image2.png with size : 2048 Directory : Documents File :Notes.txt
with size : 512 File :Report.pdf with size : 4096
```

Trace of `rootDir.display()`:

```
rootDir.display() ■■■ prints "Directory : Root" ■■■ forEach over [imagesDir, docsDir] ■■■ imagesDir.display() (Directory.display() called recursively) ■ ■■■ prints "Directory : Images" ■ ■■■ forEach over [img1, img2] ■ ■■■ img1.display() → "File :Image1.png with size : 1024" ■ ■■■ img2.display() → "File :Image2.png with size : 2048" ■■■ docsDir.display() (Directory.display() called recursively again) ■■■ prints "Directory : Documents" ■■■ forEach over [doc1, doc2] ■■■ doc1.display() → "File :Notes.txt with size : 512" ■■■ doc2.display() → "File :Report.pdf with size : 4096"
```

This is the Composite pattern's recursive power on display. The client (`main`) calls `rootDir.display()` once. Every directory at every depth traverses its own children. The indentation in the output reflects the tree depth, but the caller has no concept of depth — the recursion is entirely self-managed by the composite objects.

Notice that the driver code has zero `instanceof` checks and zero conditional logic. It simply creates the tree and calls `display()` at the root. Adding a third level of nesting (a `Backups` directory inside `Documents`) requires only adding three lines to the driver — not a single change to `Directory` or `File`.

Key Design Decisions

Why `List<FileSystemComponent>` and not `List<File>`?

```
// Correct — holds any FileSystemComponent (File, Directory, or future types) List<FileSystemComponent> fileSystemComponents = new ArrayList<>(); // Wrong — restricts children to File only; cannot contain Directory List<File> fileSystemComponents = new ArrayList<>();
```

If the field were typed as `List<File>`, a `Directory` could never contain another `Directory`, and the recursive structure of the pattern would be impossible. The interface type is the mechanism that allows the tree to be heterogeneous.

Why a method reference `FileSystemComponent::display` instead of an explicit lambda?

```
// Method reference — idiomatic, concise, no noise fileSystemComponents.stream().forEach(FileSystemComponent::display); // Equivalent lambda — correct but more verbose fileSystemComponents.stream().forEach(component -> component.display());
```

The method reference `FileSystemComponent::display` is an instance method reference on the interface type. It is a direct pointer to the `display()` method. There is no functional difference — the lambda and the method reference compile to the same bytecode behavior. The method reference is preferred because it is shorter, eliminates the need to name a parameter (`component`) that adds no information, and reads naturally as "for each element, call its `display` method."

Why `List.of()` in the driver instead of `new ArrayList<>()`?

```
Directory directory = new Directory("MyDirectory", List.of(file1, file2));
```

`List.of(file1, file2)` creates an **immutable** list. In the driver, the list of children is constructed once and passed directly to the `Directory` constructor. There is no need to add or remove elements after construction in this demo. Using an immutable list is safer — it cannot be accidentally modified through the reference returned by a hypothetical `getChildren()` getter, and it communicates intent (this list is fixed at construction).

`ArrayList` (used inside `Directory` for the field) remains appropriate for the internal storage when you want to support future `addChild()` / `removeChild()` mutations. The distinction is: `List.of()` for the construction-time snapshot, `ArrayList` as the underlying mutable storage inside the container that owns the data.

Why `ArrayList` for the internal field initialization, even though it gets overwritten?

```
List<FileSystemComponent> fileSystemComponents = new ArrayList<>();
```

The `= new ArrayList<>()` field initializer is redundant in this code because the constructor immediately overwrites it with `this.fileSystemComponents = fileSystemComponents`. However, it is a defensive coding habit: if someone were to add a no-argument constructor, or if a serialization framework instantiated the class without calling the defined constructor, the field would still be a valid (empty) list rather than `null`. Null-safe initialization of collection fields is a widely recommended practice.

The `fileSystemComponents` field is not `private` — a real encapsulation gap

```
// Current code — package-private (a bug-in-waiting) List<FileSystemComponent>
fileSystemComponents = new ArrayList<>(); // Should be private List<FileSystemComponent>
fileSystemComponents = new ArrayList<>();
```

Without an explicit access modifier, Java defaults to package-private visibility. Any class in the `com.design.patterns.composite.composite` package can read and write this field directly, bypassing any future validation or notification logic in `Directory`. This is a minor issue in a demo with only one class per package, but it would be a real encapsulation violation in a production codebase. The field should be `private`. If external access is needed, expose it through a method: either a read-only `getChildren()` returning an unmodifiable view, or `addChild()` / `removeChild()` methods that let `Directory` enforce invariants (e.g., prevent circular references).

Summary

The Composite Design Pattern solves the part-whole hierarchy problem by making composite containers and leaf nodes indistinguishable through a shared interface. The four classes in this implementation map directly to the pattern's three roles:

Role	Class	Responsibility
Component	`FileSystemComponent`	Declares the common `display()` contract
Leaf	`File`	Terminal node; fulfills the contract by printing its own data
Composite	`Directory`	Branch node; fulfills the contract by delegating to all children
Client	`CompositeDesignPattern`	Builds the tree and calls `display()` without caring about node types

The pattern's power is proportional to the depth and variety of the tree. In the flat two-file demo the benefit is modest. In a real file system with thousands of files across hundreds of nested directories — or a GUI with deeply nested panels, or a corporate org chart with five levels of management — the elimination of `instanceof`-based branching from every tree traversal operation becomes significant. The Composite pattern pushes type-awareness down into the objects themselves and out of the code that uses them.